SCHOOL OF INFORMATION TECHNOLOGY

ITWB2914 - Minor Project

A Comparative Analysis of Built-in Vs Third-Party Antivirus Software for Everyday Users: Performance, Detection, and Usability

SUBMISSION DATE:

22nd AUGUST 2025

MATRIC NO	STUDENT NAME
MC230923232	Mohamad Shazali Bin Mohamad Shaukhi

COURSE LEADER:
Dr. DANNY NGO LUNG YAO
Lecturer
School of Information Technology

SUPERVISOR:
NOOR AZMA ISMAIL
Lecturer
School of Information Technology

MODERATOR:
NOOR LEES ISMAIL
Lecturer
School of Information Technology

Table of Contents

1.1 Background of Study 5
1.2 Problem Statement 7
1.3 Project Aim 7
1.4 Project Objectives 8
1.5 Project Scope 8
1.6 Target Audience 9
1.7 Project Justification 10
1.8 Project Timeline 11
2.1 Introduction 12
2.2 Theoretical Framework 12
2.3 Review of Related Works 13
2.3.1 Malware Detection Effectiveness 13
2.3.2 System Performance Impact 14
2.3.3 Usability and User Perception 14
2.3.4 Classification of Antivirus Software 15
2.3.5 Existing Benchmarking Tools 15
2.4 Critical Analysis of Existing Literature 16
2.5 Emerging Trends in Antivirus Technologies 18
2.5 Summary of Findings and Research Gaps 18
3.1 Introduction 20
3.2 System Development Model 20
3.3 System Requirements and Specifications 21
3.3.2 Non-Functional Requirements 22
3.3.3 Hardware and Software 22
3.3.4 Constraints 23
3.4 Methodological Approach 23
3.5 System Architecture / Framework 24
3.6 Implementation and Development Process 26
4.1 Introduction 28
4.2 System Requirements Overview 28
4.2.1 Functional Requirements 28
4.2.2 Non-Functional Requirements 29
4.3 System Design Evaluation 29
4.4 Usability Analysis 30

4.5 Anticipated Challenges and Limitations.....30
4.6 Future Improvements.....31
5.0 Introduction.....32
5.1 System Architecture & Modules.....32
5.2 Interface Design.....33
5.3 Data Design.....33
5.4 Technology, Security & Performance.....34
5.5 Error Handling, Deployment & Maintenance.....35
5.6 Design Challenges & Summary.....36
5.7 UI Design Mockup.....36
6.0 Conclusion.....46

CHAPTER ONE

INTRODUCTION

The emergence of early antivirus software back in the 1980s sparked the birth of a new industry where early developers of third-party antivirus solutions such as McAfee and Norton first emerged as pioneers of the commercial antivirus industry. The “Brain” computer virus in 1986 which was the first computer virus targeted at IBM PCs pushed the development of anti-virus software to combat these threats (Echegu & Agwu, 2024).

The creation and widespread usage of the World Wide Web in the 1990s directly led to the emergence of more sophisticated viruses when compared to their predecessors. During this era, the creation of Macro viruses targeted computer software such as Microsoft Word to infect and spread via email attachment and caused significant damage by overwhelming email servers (Nilupul, 2024). As a countermeasure, antivirus companies such as McAfee and Norton constantly pushed updates to protect their users against any dangerous viruses and became synonymous with basic computer protection especially for home and business users. This made third-party antivirus software a household name and cemented their reputation for protecting everyday users from cyber threats.

As malware continue to evolve and threaten everyday users with increasingly advanced threats beyond traditional exploits and viruses, it is no wonder that third party antiviruses have managed to maintain their reputation. The study by Ferdous (2023) highlighted a sharp rise in ransomware, cryptojacking and fileless malware which operates entirely in RAM alongside attacks targetting mobile devices, IoT networks and remote workers. This rise in increasingly dangerous cyber-attacks means users have to be weary of their devices being hijacked, performance degradation and in the worst case scenario data theft and financial loss all without visible signs. With the most popular method of attack being phishing, everyday users are at risk of exposing themselves to these threats during their daily browsing.

However, as operating systems continue to develop, so do their built-in security services and protection features, for example, Microsoft Defender. With modern operating systems such as

Windows 11 offering a solid and capable built-in antivirus solution, the question arises whether there is still a need to purchase and invest in third-party antivirus solutions (Delaney, 2020). Despite Microsoft Defender's evolution as a reliable antivirus solution, Cruz (2024) reported that over 121 million adults rely on third-party antivirus software. Americans aged 65 or older were twice as likely to pay for antivirus services compared to those under 45. While over half of users stick with system default antivirus or none at all, nearly half still rely on third-party solutions which indicates a continued belief in their added value. Of those who use third-party antivirus, 54% pay for the service, with 84% of users citing general security against malware and viruses as their primary reason for using third-party antivirus.

With built-in antivirus like Microsoft Defender improving significantly over the years, many users face a dilemma: is investing in third-party antivirus software still worthwhile, or is the default protection provided by the system now sufficient? This project aims to compare and evaluate the performance, detection, and usability of built-in antivirus solutions and third-party antivirus software in Windows 10 and 11 environments. This will be done by reviewing the functionality of several major third-party antivirus solutions and Microsoft Defender. The results provided from this project will determine whether there is value in using third-party antivirus software by comparing both antivirus solutions' performance and responsiveness to simulated threats. The project involves developing a lightweight computer software program that uses a standardized test to simulate malware detection capabilities of a third-party antivirus by measuring system performance during the detection process and comparing it against Microsoft Defender as the benchmark to determine whether the average user still benefits from third-party antivirus tools or if Microsoft Defender now provides sufficient protection by default for everyday users.

1.1 Background of Study

Microsoft Defender formerly known as Microsoft AntiSpyware was first released on October 24, 2006 and was launched as an anti-spyware tool for Windows XP and Windows Server 2003. Over time, it was developed into a full-fledged antivirus program and was completely implemented as Windows operating system built-in antivirus with Windows 8. It aimed to provide features such as real-time protection against malware and spyware as well as other known software that can potentially damage the operating system (Shinder, Diogenes, & Shinder, 2013).

On the other hand, third-party antivirus software has been offering real-time protection, prevention, performance enhancement, and safeguarding of data as early since its inception in the 1980s. Combined with the industry's commitment of updating regularly to address new security issues, introduce new features, correct bugs and so on makes owning and paying for a third-party antivirus software more enticing (Echegu & Agwu, 2024). These offerings of new features often claim superior malware detection capabilities, smarter threat intelligence and customization features that go beyond what built-in antivirus solutions typically provide.

Despite that, in a report by Cruz (2024) 40% of Americans specifically opt to use system default antivirus with the reasoning of not needing a third-party antivirus. Paired with the fact that 54% of Americans use system default antivirus, this shows a strong trust in the capabilities of the built-in antivirus in defending their system to the point where they do not feel the need to explore and try out a third-party antivirus.

While third-party antivirus software does offer advanced and up to date security features such as McAfee Total Protection providing anti-phishing capabilities and Norton Antivirus offering features such as identity theft protection, some users view these additions as unnecessary or excessive. The he constant updates, complex interface, and bloatware that comes with the software makes it a hassle for users who rarely encounter security threats in their daily usage.

Moreover, concerns around data privacy and pricing have caused further skepticism. Cruz (2024) reported that many non-users to have security concerns regarding the misuse of customer data as evidenced by Avast deceptive practices in collecting and selling detailed user browsing data to third parties. On top of that the high price point of third-party antivirus software sits at the top of the reason as to why users do not use antivirus software. These factors acts as a significant barrier for casual users who may or may not need the additional features.

Therefore, based on the differences and user preferences, it can also be said that it is essential to evaluate antivirus software based on three core aspects, performance, detection and usability. Performance refers to how efficiently the antivirus runs in the background of the system and its impact on the performance of system processes and resource usage. Detection is the software's ability to accurately identify and respond to various types of malware while also minimizing false positives. Usability covers on how intuitive and user-friendly the software is which includes its interface design, notification handling, ease of navigation and setup process. These three core

aspects directly impact the overall effectiveness and usability of an antivirus solution especially for everyday users who prioritize smooth system performance, reliable protection and a hassle-free user experience.

By objectively comparing antiviruses performance based on system performance, malware detection accuracy and overall usability by using a lightweight and easy-to-use evaluation tool, it enables users to make informed and factual decisions based on their antivirus needs whether it be a user seeking to evaluate their current antivirus or a family member assisting their family members.

1.2 Problem Statement

As third-party antivirus solutions continue to develop and compete with an increasingly improved built-in antivirus solution such as Microsoft Defender, there remains uncertainty about which option offers more protection, efficiency and usability for the everyday user. While the study conducted by Baker (2018) measured the impact of third-party antivirus software on the system performance compared to Microsoft Defender, the study did not test detection rates, virus scanning speed and malware removal capabilities which is an aspect this project focuses on.

Most home users rely on marketing claims or the popularity of third-party antivirus solutions leading to unnecessary spending, reduced system performance and a false sense of security especially if the software has features that the user may not even need.

While advanced benchmarking and antivirus testing platforms do exist, they are often intended for industrial experts. There should be a lightweight, user-friendly software with the purpose of assisting users in determining whether or not their antivirus is good enough for their day to day usage. The software should be backed by reliable tests conducted in a safe environment to give the benchmark tool credibility in its comparison.

1.3 Project Aim

This project aims to answer the following questions.

PQ1: Are third-party antivirus solutions better in terms of detection performance, system impact, and usability compared to built-in antivirus like Microsoft Defender for everyday users?

PQ2: Are third-party antivirus solutions still relevant or necessary for the average user in 2025, given the improvements in default operating system security?

PQ3: Can a reliable dataset of antivirus performance against standardized threat simulations be created to support objective benchmarking and comparison?

PQ3: Can a lightweight, user-friendly benchmarking tool be developed to help users make informed decisions by objectively comparing antivirus performance and detection responsiveness?

1.4 Project Objectives

This project aims to achieve the following objectives.

PO1: To determine the difference in detection performance, system impact and usability of third-party antivirus solutions compared to Microsoft Defender in a Windows 11 environment.

PO2: To determine the relevance and necessity of owning a third-party antivirus solution in 2025.

PO3: To develop an automated tool that tests antivirus performance against standardized threat simulations in a closed environment and compiles the results into a dataset for benchmarking purposes.

PO4: To develop a lightweight and simple antivirus benchmarking tool to assist users in making informed decisions on the performance of their antivirus.

1.5 Project Scope

This project will provide a comprehensive analysis of built- in vs third-party antivirus software for everyday users. The report will cover antivirus technologies and types with the intention to clearly understand how different antiviruses differ in their detection techniques. This includes the features of several major third-party antivirus solutions and Microsoft Defender. Other areas of focus in this report will be performance indicators of antivirus software, benchmarking and comparative analysis methodologies and the limitations and challenges in assessing antivirus solutions and its security effectiveness.

The project will only cover commercial antivirus products aimed for everyday users and not cover enterprise or server-level antivirus platforms. This project will also only focus on the Windows 11 operating system and not evaluate other platforms such as mobile platforms, Linux and macOS.

The lightweight benchmark tool will only run on 11. Its purpose is to automatically detect the current antivirus used by the system and compare it with results stored inside the tool. The user

will be presented with a GUI that compares all available antivirus with parameters such as detection rate, average detection time, CPU usage, memory usage and a total score out of 100. This dataset will be collected via a separate program developed to test the antivirus capabilities against standardized threat simulations in a closed environment.

The data collection tool will be developed in python to automatically run PowerShell commands in a safe environment to test the antivirus capabilities in real time and generate a report in the form of a .txt file. This report will then be compiled with all other tested antiviruses and become the dataset for the benchmark tool.

The data collection tool includes using PowerShell commands to generate an EICAR file to ensure the antivirus is working as intended first. Once the antivirus has been confirmed to be working, the tool will use PowerShell commands to run Atomic Red Team tests which will be bundled together with the tool to ensure it works in an offline environment. The tool will then measure tests detected, detection rate, average detection time, average CPU usage, average RAM usage and produce an overall score using the weighted scoring method. The tool will then store all information measured into a .txt file.

The data collection tool will only be used on a controlled safe environment which is a windows 11 virtual machine using Hyper V.

1.6 Target Audience

This project is targeted towards the following audience.

TA1: Everyday Windows users who want to understand the effectiveness of their antivirus software without technical expertise.

TA2: Tech-savvy family members or caregivers who assist less experienced users, such as older adults, in making informed decisions about antivirus protection.

TA3: Windows users aged 18 to 40 who are interested in assessing the need for third-party antivirus solutions versus built-in protection.

1.7 Project Justification

This project aims to clearly compare third-party antivirus solutions with the built-in antivirus solutions packaged together with the Windows 11 operating system. By doing so, it allows users to make informed decisions regarding their usage of antiviruses. This project aims to inform users on how well their antivirus works by conducting a comparative analysis of third-party antivirus software against built-in, supported by a purpose-built software tool designed to measure the performance of each antivirus in a controlled environment.

The significance of this project lies in its purpose to address and determine whether investing in a third-party antivirus software is necessary for users who are either thinking of using a third-party antivirus software or is currently using one. Whether it be to save cost or to simply get rid of annoying bloatware, this project aims to inform users of the capabilities of their current antivirus when compared to others on the market. With many users relying on marketing claims or assumptions, there is a critical need for a data driven project into the real-world effectiveness and impact of these antivirus solutions.

The project contributes to the field by developing two separate software that appeal to both enthusiast and casual users of computers. The data collection tool is an automated antivirus performance tester which measures the antivirus capability to deal with standardized threat simulations while the benchmark tool is a straightforward easy to use software that anyone can launch on their windows 11 operating system to measure their currently in use antivirus solutions against other antiviruses based on a carefully measured dataset collected by the data collection tool.

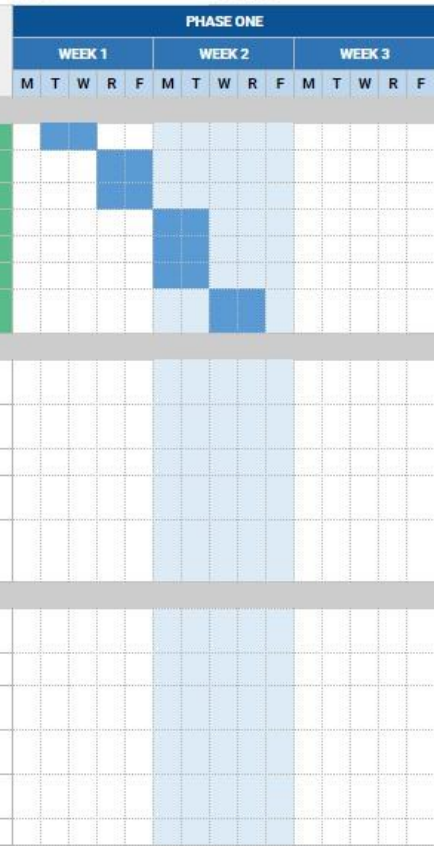
The project dives deep into the performance and usability of current available antivirus solutions and further explores these aspects through the development of a dedicated benchmarking tool and data collection tool.

1.8 Project Timeline

GANTT CHART

PROJECT TITLE	A Comparative Analysis of Built-in vs Third-Party Antivirus Software for Everyday Users: Performance, Detection, and Usability				
PROJECT MANAGER	Mohamad Shazali Bin Mohamad Shaukhi			DATE	26/5/25

WBS NUMBER	TASK TITLE	START DATE	DUE DATE	DURATION (DAYS)	PCT OF TASK COMPLETE
1	**Chapter 1: Introduction**				
1.1	Write Introduction	27/5/25	28/5/25	1	100%
1.2	Write Problem Background	29/5/25	30/5/25	1	100%
1.3	Write Problem Statement	29/5/25	30/5/25	1	100%
1.4	Establish Project Aim & Objectives	2/6/25	3/6/25	1	100%
1.5	Establish Project Scope & Audience	2/6/25	3/6/25	1	100%
1.6	Write Project Justification	2/6/25	3/6/25	1	100%
1.7	Create Gantt Chart for Project Timeline	4/6/25	5/6/25	1	100%
2	**Chapter 2: Literature Review**				
2.1	Write Literature Review Intro and Define Theoretical Framework	9/6/25	11/6/25	2	0%
2.2	Review Related Studies (Baker 2018, Delaney 2020)	12/6/25	13/6/25	1	0%
2.3	Critique the Related Studies	16/6/25	17/6/25	1	0%
2.4	Address Emerging Trends and Developments in Antivirus Technology	18/6/25	19/6/25	1	0%
2.5	Summarize and Identify Gaps; Link to Research Questions	18/6/25	19/6/25	1	0%
3	**Chapter 3: Methodology**				
3.1	Draft Methodology Introduction & Development Model	20/6/25	23/6/25	3	0%
3.2	Define Requirements & Constraints	24/6/25	25/6/25	1	0%
3.3	Describe Research/Development Approach & Tools	26/6/25	30/6/25	4	0%
3.4	Document System Architecture & Future Plan	26/6/25	30/6/25	4	0%
3.5	Document System Architecture & Interaction Flow	1/7/25	3/7/25	2	0%
3.6	Identify Future Implementation Steps	1/7/25	3/7/25	2	0%
4	**Chapter 4: System Analysis**				
4.1	Analyze System Requirements & Design	4/7/25	8/7/25	4	0%
4.2	Describe Expected Performance & Usability Plan	9/7/25	11/7/25	2	0%
4.3	Identify Security, Privacy, & Constraints	14/7/25	15/7/25	1	0%
4.4	Discuss Challenges & Recommendations	16/7/25	17/7/25	1	0%
5	**Chapter 5: System Design**				
5.1	Document System Architecture & Data Design	18/7/25	21/7/25	3	0%
5.2	Plan UI & Functional Module Design	22/7/25	25/7/25	3	0%
5.3	Plan Tech Stack, Tools, & Deployment Strategy	22/7/25	25/7/25	3	0%
5.4	Plan Security, Scalability, and Logging Features	22/7/25	25/7/25	3	0%
5.5	Identify Design Challenges & Future Maintenance Plan	28/7/25	31/7/25	3	0%
6	**Report Finalization & Submission**				
6.1	Receive Presentation Schedule	13/8/25	13/8/25	0	0%
6.2	Minor Project Presentation	14/8/25	14/8/25	0	0%
6.3	Report Corrections	15/8/25	15/8/25	0	0%
6.4	Final Report Submission	18/8/25	22/8/25	4	0%



10CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

A comprehensive review of existing literature is important in order to fully gauge and understand the capabilities of existing antivirus software and malware. To understand the evolution of malware and the improvements of antivirus software to combat such cyber threats, a thorough review of existing literature will give an insight on the development, evaluation and current capabilities of antivirus solution in the context of combating cyber threats.

This chapter is organized thematically to cover key areas of antivirus software evaluation which includes system performance, threat detection. However, before delving into reviewing the following studies, this chapter will first outline the theoretical frameworks that will support the study's focus on cybersecurity effectiveness and user-centered evaluation.

2.2 Theoretical Framework

The key theories and concepts this study draws from are several foundational models and frameworks relevant to cybersecurity usability, performance evaluation, and user behavior. These include the Technology Acceptance Model (TAM), the Information Systems Success Model, and Performance Benchmarking Theory, all of which have been widely applied in the field to explore the adoption and effectiveness of software solutions.

The Technology Acceptance Model (TAM) offers insights into how users perceive and accept antivirus software, especially in comparing free built-in solutions like Microsoft Defender with paid third-party alternatives. The Information Systems Success Model on the other hand focuses in understanding user satisfaction, system quality, and perceived usefulness with key aspects such as evaluating antivirus usability and value.

Furthermore, Performance Benchmarking Theory provides a structured base for measuring the operational impact of antivirus software on system resources, such as CPU usage, memory consumption, and detection responsiveness.

These theories assist the research by providing a structured view through which the project's core questions related to performance, detection, and user decision-making can be examined and studied. By applying these theoretical frameworks, the study aims to explore whether third-party antivirus solutions still offer benefits over Microsoft Defender in 2025, ensuring that the analysis is grounded in established perspectives and informed by previous research in both cybersecurity and user-centered software evaluation.

2.3 Review of Related Works

Each of the selected studies will be reviewed based on the following category that has been assigned to each one. The Effectiveness of Antivirus Software by Jeffrey Delaney (2020) will be carried out focusing on the study's elaboration on malware detection effectiveness. The Necessity of Third-Party Antivirus Software by Erik Baker (2018) studied the impact of antivirus software on system performance in depth, therefore, a review will be done focusing on that aspect. Lastly, From Origins to Future Trends by Arinze Darlington Echegu (2024) and other relevant literature works will be reviewed to study how users perceive antivirus tools.

2.3.1 Malware Detection Effectiveness

Detection capability remains the most crucial function in antivirus evaluation therefore, the efficiency of an antivirus solution in detecting malware is arguably the most important feature in an antivirus software. Based on the research conducted by Delaney (2020), various detection techniques were examined that were used by antivirus solutions. Signature-based detection was discovered to only be effective against known cyber threats, while on the other hand heuristic methods and behavioral analysis aimed to overcome that limitation and adapt to new threats. Although heuristic detection is able to spot unknown threats, it also tends to trigger false positives whereas behavioral detection only observes software activity in order to identify suspicious or unusual pattern behavior making it more effective against malware that is stealthy. Another thing that was noted in the study was that the emergence of next generation antivirus (NGAV) utilizes a real-time endpoint monitoring and AI-driven models to assist with detection and response.

In a real world environment test, the study showed that only half out of the 30 antivirus products tested by AV-Comparatives met the minimum 98% detection rate which indicates that many antivirus solutions are not up to standard. The study also focused on Windows Defender with it

showing strong results in zero-day protection especially since it applies NGAV technology however, it was also found that it still lagged behind other top tier products in areas such as adware detection. The conclusion of the study was that antivirus tools are still relevant but has to constantly evolve to keep up with modern cyber threats. These findings further prove the necessity of further evaluating the comparative strength of built-in solutions like Windows Defender.

2.3.2 System Performance Impact

It is to be expected that installing a third party antivirus solution would affect systems performance as having an always running program in the background of the system is bound to affect system performances. In the study conducted by Baker (2018), it was shown that using third party antivirus solutions in an offline environment had minimal impact on system performance, moreover it was discovered that third party antivirus solutions had a positive impact in systems performance particularly when it came to loading and saving documents. In the study it was concluded that based on the context of system performance, in a Windows 10 environment a third-party antivirus solution allows certain tasks to go faster by a few seconds despite having multiple detection methods running in the background. This highlights the importance of comparing and measuring performance tradeoffs especially in the context of non-technical users choosing between free and paid antivirus.

2.3.3 Usability and User Perception

Usability is an important aspect when discussing the relevance of antivirus software as it can be said that the primary users of antiviruses are everyday users. This highlights the significance of accessible interfaces, minimal system disruption and clear communication of threats and actions. As stated in Echegu & Agwu, (2024) user engagement and awareness significantly influence the effectiveness of security practices which is completely in line with the Technology Acceptance Model which emphasizes the perceived ease of use and usefulness in determining user adoption. These insights establish the conclusion that antivirus usability should not be overlooked, as users seek a solution that they can easily understand and learn which in turn directly impacts the overall security of individuals online.

2.3.4 Classification of Antivirus Software

To further define and separate built-in and third-party antivirus, the following table summarizes their general differences across key differences that are relevant to everyday users.

Feature	**Built-in**	**Third-Party AV**
Default Installation	Yes	No
Price	Free	Free or Paid
Real-Time Protection	Yes	Yes
Ransomware Protection	No	Yes
Firewall Protection	Yes	Yes
System Resource Usage	Low - Medium	Medium – High (Varies)

The following classification table in 2.3.4 was based on comparative insights drawn from the previously mentioned and reviewed literature and general product documentation. The studies reviewed will directly influence the benchmarking process and evaluation criteria in the following chapters, guiding both the technical assessment and user-centered comparison of antivirus performance.

2.3.5 Existing Benchmarking Tools

In addition to academic literature, there exists independent organizations such as AV-Comparatives, AV-Test and SE Labs which conducts large-scale antivirus benchmarking. For example, AV-Comparatives recently published a report regarding Real-World Protection Test (February – May 2025) which assessed 19 consumer security solutions and evaluating their protection performance under realistic conditions which included malicious URLs, downloads and zero day threats which produced a result of Bitdefender achieving the top protection rate of 99.8% while Microsoft Defender managed to produce a protection rate of 99.1% (AV-Comparatives, 2025). This real-world benchmarking underlines that built-in antivirus capabilities have dramatically improved and can compete with third party antivirus solutions. However, these tests focus on purely on technical aspects of antiviruses and largely ignore usability factors. Another thing to consider is that the testing methods is not made available to

the public and is only defined in the report. Thus, while independent organizations such as AV-Comparatives establishes industry-standard baselines, this project differentiates itself by combining detection metrics with usability and performance trade-off analysis while ensuring its transparent and open source tailoring to everyday users choosing between built-ion and third party antivirus solutions.

2.4 Critical Analysis of Existing Literature

The studies reviewed have significantly produced results that have contributed to the understanding of antivirus in terms of detection capabilities, system performance and user perception. Delaney (2020) study provided a comprehensive evaluation of malware detection effectiveness by drawing from sources such as AV-comparatives. The study highlighted that while traditional signature-based detection remains in use by antivirus solution, its limitation against the ever growing threat of cyber threats has pushed antivirus companies to pursue the adoption of next-gen AI-based detection methods. The study excels in highlighting the constant evolution of malware and the necessity for dynamic antivirus responses however, while it is informative it falls short in addressing the practical usability and applicability of antivirus software for everyday users. The study weakness can be seen in its dependence on lab-controlled AV-Comparatives which

while it provides sufficient context for IT professionals or organization, it lacks on providing sufficient analysis on user interaction and overall system usability.

The study conducted by Baker (2018) on the other hand, analyzes how third-party antivirus software affects system performance while in the background. Contrary to common belief, the result of the study suggested that certain third-party antivirus actually enhance task efficiency under specific conditions such as offline usage. The gap in the study can be seen in its lack of detailed hardware or sample size information which limits applicability and its complete omission of detection capabilities of antivirus solutions with the study putting more focus on only system performance impact.

In the research conducted by Echegu and Agwu (2024), the detailed breakdown on how malware evolves, adapts and bypasses traditional detection mechanisms such as signature based detection. The research goes in depth on advanced methods like sandboxing and behavioral analytics and puts a strong emphasis on malware behavior and the capabilities of modern protection techniques however, it offers very little discussion on how these advancements translate to actual improvements for the everyday users. It does not explore the usability or accessibility of these technologies for non-technical users who are often the primary targets of malware campaign. Combined, these studies reflect a broader trend where built-in antivirus solutions have significantly improved making it close the performance and detection gap with a majority of third party antivirus. However as stated before, third-party antivirus continues to differentiate themselves through premium features which built-in antivirus may or may not have. Despite each literature's strength, none of them directly compares the real-world performance and usability of built-in antivirus and third party antivirus especially from the perspective of an non-technical user.

Consequently, these studies while insightful do not fully focus on users who are often the main victims of cyber threats and are also the main consumers of antivirus software. This research aims to fill these gaps by focusing on third party antivirus software relevance to non-technical users with the existence of a capable built-in antivirus solution such as Microsoft Defender.

2.5 Emerging Trends in Antivirus Technologies

While it has been established that antivirus software uses signature based detection to counter cyber threats, its inability to detect and respond to new threats in real time pushed the industry to evolve and develop NGAV. Organizations such as IBM, Microsoft and Threatdown has steadily implement NGAV into their antivirus solutions to further improve their defenses against cyber threats. The main advantage of using NGAV is that it does not rely on malware signature but rather it uses machine learning, behavioral analytics and exploit prevention to fight against malware in real time (Lindemulder & Forrest, 2023). Aside from that, many NGAV systems now utilize cloud based techniques to provide real time detection updates and reduce local system impact. For example, Microsoft utilizes NGAV in their Microsoft Defender for Endpoint offering key features such as Endpoint Detection and Response (EDR) which continuously monitors device activity and detects suspicious behaviors in real time (denisebmsft, 2025). While Microsoft has steadily updated their Microsoft Defender with NGAV technology, other third-party solutions such as Norton has also adopted similar improvements into their antivirus solutions often packaged as premium features. This key difference highlights different priorities in accessibility, pricing models and user target groups.

Despite all the advanced features NGAV has to offer, it has several significant challenges especially for the average user. NGAV uses more resources compared to traditional antivirus solutions which might not be worth the trouble for everyday users and on top of that it is still not a foolproof solution to combatting malware. NGAV technology should be used as a supplement alongside other antivirus technology as even NGAV cannot produce a 100% success rate without raising false positives and failing to detect a virus. Therefore, its real world effectiveness still hinges on how well it balances security with usability and system performance.

2.5 Summary of Findings and Research Gaps

The literature reviewed has established that antivirus software continues to hold a critical role in protecting users from cyber threats. Studies such as Delaney (2020) have proven that some third party antivirus solutions still offer strong detection capabilities due to their advanced features like machine learning. Additionally, the study done by Baker (2018) has shown that third party

antivirus software can actually further improve system performance in tasks such as file loading. Lastly, Echegu and Agwu (2024) reported that user awareness and usability directly impacted the overall effectiveness of antivirus solutions.

However, there are still gaps and questions that remain unanswered when reviewing the following studies. For example, Delaney (2020) comprehensively reviewed and evaluated detection methods of antivirus solutions and the performance of Microsoft Defender, but the study does not explore how non-technical users perceive its usability or whether its limitations justify purchasing third-party solutions for everyday needs. Echegu and Agwu (2024) did not focus on the role of antivirus software protecting everyday users who are most often the main target of malware attacks. Baker (2018) only focused on benchmarking the effects of third party antivirus software on system performance and did not run benchmarks or evaluate it based on factors such as usability and detection rate. These limitations highlight the need for further research that not only compares antivirus effectiveness but evaluates practical usability and relevance for day to day users.

Therefore, it can be established that there is a significant lack of research that benchmarks built-in and third-party antivirus software in a single framework focusing on both aspects of system performance impact and malware detection capabilities especially from the perspective of a casual and non-technical user. Furthermore, time has significantly passed that may change the results of their findings due to how fast technology develops. 5 years is plenty of time for built in antivirus to improve their performance. This project will seek to address these gaps by providing an analysis on both built-in antivirus solutions and third party using custom testing and structured evaluation criteria tailored to both technical performance and end-user experience

CHAPTER THREE

METHODOLOGY

3.1 Introduction

This chapter will outline the approach, techniques and procedures which will be used in this project to design and develop the system. The benchmark tool system aims to benchmark antivirus software through factors such as effectiveness and practicality in context of everyday users. The data collected from the benchmark system will be fed into the comparison system to compare and evaluate the user's current antivirus with other antivirus solutions based on the overall score collected from the benchmark tool. The evaluation will be guided by the Technology Acceptance Model (TAM), Information Systems Success Model (ISSM) and Performance Benchmarking Theory. As for the overall score which will be used to compare each antivirus performance, a weighted scoring method will be implemented to provide an overall score based on detection, performance impact and usability.

The approach of creating two separate tools is to ensure that antivirus feasibility testing can be done in a safe environment and does not jeopardize the user's system. The TAM is important to explain how users accept and utilize new technologies which in this case will be focused on how users accept and use the latest antivirus technology available. This is to highlight the usability aspect of this project. The ISSM on the hand is used to focus on the technical aspects of the system such as the functionality, ease of use and reliability. By providing a solid benchmark score for each antivirus and providing users with the ability to view how their antivirus performs against other antivirus solutions, users are able to make an informed decisions regarding their usage of antivirus solutions. Throughout the development of the systems, the agile methodology will be applied due to its advantage in flexibility, and adaptability.

3.2 System Development Model

The chosen system development model for this project is the Agile methodology. This methodology suits this project best due to the fact that the systems will require incremental improvements, flexibility and iterative testing. When testing the antivirus effectiveness via the

benchmark model, it will involve multiple variables such as types of malwares, system performance which makes a rigid model such as Waterfall methodology unsuitable.

Agile methodology is a methodology that strives in its ability to shift strategies quickly, making it a perfect fit for this project. Because of that, it allows the project to change and continuously improve until the end of planning and development. The main phases within the agile methodology includes planning where requirements are set, design of the features of the project, implementation where the development is conducted, testing to ensure the project meets the requirements and lastly review of the work completed. One of the key principles of Agile is to present a working outcome at the end of each phases such as a working test module.

The first phase in the Agile methodology will be planning, in which the test objectives will be established which includes variables such as detection accuracy and speed, system performance and phishing resistance. Antivirus solutions to be tested will also be chosen in this phase based on its popularity and reputation and lastly, the virtual machines will be established and a home lab system will be created to house the virtual machines for a closed and safe environment for testing.

The next phase is the design phase in which the development of the test environments and the automation scripting via python for the phishing simulation, EIRCA deployment, the atomic red team tests and system monitoring to track CPU and RAM usage during testing.

Following that, the next phase is the implementation phase where the benchmark tool will be deployed on each AV within isolated VMs. The detection behavior will be recorded as well as how many tests were successfully run, phishing response will also be recorded and system metrics will be recorded.

The last phase is the evaluation phases in which the results from the testing will be automatically scored using the weighted scoring method based on detection, usability and performance to produce an overall score for each antivirus software. If any inconsistencies are found or test failures occur, the tests will be refined and adapted to ensure the results are credible.

3.3 System Requirements and Specifications

3.3.1 Functional Requirements

The benchmark tool will be able to:

- Automatically generate the EICAR test file within the virtual machine.
- Simulate a phishing campaign using the GoPhish framework to send test emails with malicious links and attachments and automatically open the link to trigger the antivirus response.
- Deploy Invoke Atomic Tests bundled with the program to run 10 preselected tests.
- Record antivirus detection and system performance metrics in real-time during all testing phases.
- Automatically generate a report in .csv format based on the data collected.
- Calculate an overall score using the weighted scoring method based on detection, usability, and performance.
- Display the results in the comparison tool's Python-based GUI.
- Detect the currently installed antivirus through PowerShell and include its performance in the comparison results.

3.3.2 Non-Functional Requirements

The system will have the following qualities:

- All testing will be consistent and repeatable for every antivirus test.
- The same methods (EICAR, GoPhish, Invoke) will be used for all antivirus solutions to provide a consistent benchmark.
- Each virtual machine will have the same setup to ensure identical testing conditions.
- Detection events, timestamps, and system performance metrics will be recorded in real-time.
- The design will be scalable to allow more antivirus solutions to be tested in the future without additional development.
- The test environment will reflect a typical user's computer rather than a high-end lab setup.
- Testing will be carried out in an isolated environment to prevent risk to the host system.

3.3.3 Hardware and Software

The testing will be hosted on a Homelab with the specs of:

CPU	RAM	STORAGE	MOTHERBOARD	GPU	OS
AMD Ryzen 5 3600 (6 Cores, 12 Threads 3.6GHz	16GB DDR4	500GB NVMe SSD	A520M chipset with virtualization support	AMD R5 430 2GB GDDR5	Proxmox VE 8.x running directly on bare metal

There will be a total of 11 virtual machines with 1 hosting the GoPhish framework and the other 10 outfitted with 10 different antivirus solutions where the Microsoft Defender will act as the baseline for comparison. Each VM will have the same setup with CPU set to 2-4 vCPUs, RAM set to 4 – 8 GB, Storage set to 60GB and network set to NAT with internet access. Before each AV

installation, a clean VM snapshot will be taken to ensure a consistent test environment. Each AV will be testing in an isolated VM to ensure no interference and the baseline VM will be the one with only Microsoft Defender as its AV.

Tools that will be used in this project will be Proxmox VE to manage and host all VMs in the homelab. GoPhish server to simulate phishing attacks. Atomic Red Team via Invoke-Atomic to simulate malware attacks. Windows Performance Monitor to log CPU usage, RAM usage. Python scripts to automate logging, extracting timestamps and automatically generating reports.

The ten antivirus solutions that will be tested is Microsoft Defender, Bitdefender Antivirus, Kaspersky Security Cloud, Norton 360, McAfee Total Protection, Avast Antivirus, AVG Antivirus, EST NOD32 Antivirus, Trend Micro Antivirus+ and lastly Malwarebytes.

3.3.4 Constraints

AV licensing restriction is the first constraints of this project which limits the number and type of third party antivirus software that can be legally installed and tested within the environment. Some commercial AV products may require paid licenses or trial limitations which restricts the duration and scope of testing.

Secondly, another constraint of this project would be the fact that limitations in malware testing exist in which there is a lack of safe and controlled tools suitable for everyday users. Aside from that, due to safety concerns and ethical guidelines, real malware samples cannot be used in testing environment and it would also hold little credibility due to its signature already being tagged by most AV solutions. As a result, simulations are dependent on test files such as atomic red team techniques which may not fully reflect real-world malware behaviors but ensure safety and replicability.

3.4 Methodological Approach

This project will utilize the quantitative experimental approach with a structured and repeatable testing framework. The reason this approach was chosen is due to the fact that the system will be designed using a modular, script-driven methodology where automation and consistency is the top priority during development. Testing will be carried out within isolated virtual environments using the Windows 11 operating system and hosted by Proxmox as the virtualization platform to ensure

hardware uniformity. As for the evaluation phase, the system is composed of three core modules which is the antivirus status phase using the EICAR test files, a phishing detection phase using GoPhish server integration and lastly a malware simulation phase using the Atomic Red Team tests via Invoke. All tests will be executed automatically using a combination of Python scripts, PowerShell, and Windows Performance Monitor and the results will be recorded and processed into the Comparison system developed using Python to display and organize the data collected.

In order to generate a credible comparative score relevant to everyday users, a weighted scoring system will be used where the values will be set as 50% for malware detection, 30% for system performance, and 20% for usability factors. This is important to ensure that an AV solution that completes the testing quickly despite failing most test will not return a high score during testing. Malware detection is evaluated based on antivirus responsiveness and coverage across all testing phases and is set to 50% due to this projects focus on testing the effectiveness of AV solutions in protecting everyday users. Performance metrics such as CPU usage and RAM consumption will be recorded during these tasks and is set to 30% due to its most obvious impact to an everyday user when using a system with an AV installed. Lastly, usability will be evaluated based on interface clarity, installation process and feature accessibility. This setup reflects structured evaluation principles as cited in (Drummond & Murphy-Reyes, 2018) where it emphasized consistent test environments replicable procedures and transparency in scoring.

3.5 System Architecture / Framework

The benchmarking tool is structured around three core modules, the malware detection simulation, which includes the use of the EICAR test file and Atomic Red Team test files, the phishing simulation which utilizes the GoPhish framework and the performance and usability logger. These modules are designed to work together in an automated and virtualized testing environment. The first two modules simulate real-world threat scenarios to measure antivirus responsiveness to malware and phishing attempts. The final module records system performance using Windows Performance Monitor and Python-based logging scripts that automatically track system impact throughout all phases of testing.

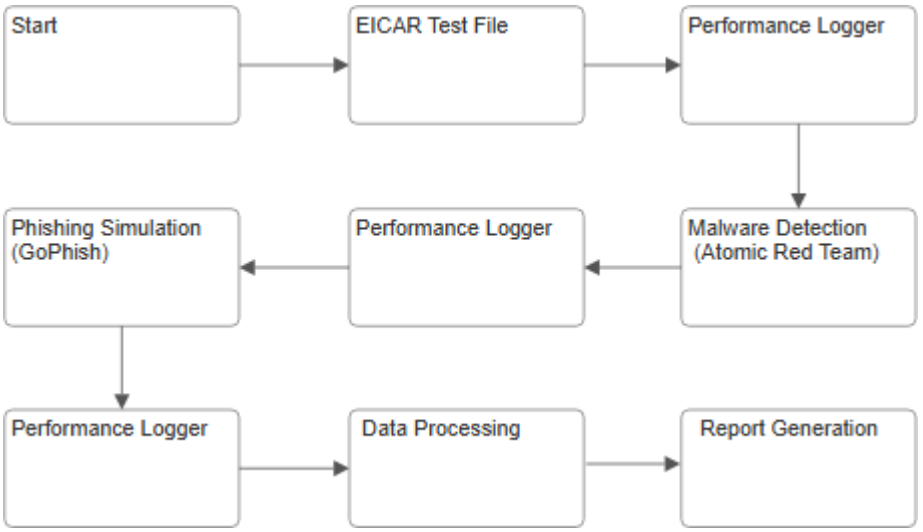


Figure 0.1 Block Diagram

Each component interacts seamlessly in a linear testing flow. The EICAR file simulation is executed first by automatically generating and placing the test file into the virtual system. The program logs how long the antivirus takes to detect and remove the file, confirming the AV's real-time protection functionality. Once verified, the GoPhish simulation begins. In this phase, the program running in the Test VM initiates a script that calls the HQ VM which hosts the GoPhish server to send a phishing email to a preconfigured address, such as a disposable Gmail or Outlook account created solely for testing. Once the phishing email is received inside the Test VM, the script automatically opens the phishing link embedded in the email, simulating a user interaction to trigger a phishing attempt. The final module runs 10 Atomic Red Team tests, which are dropped sequentially into the system. All test interactions and detections are logged in real-time, capturing detection timestamps, system impact, and AV behavior.

The data flow begins with input from the testing modules, where detection data and system metrics are continuously recorded. This data is processed and stored in structured log files, which are then exported to a separate Python-based comparison program. This GUI tool automatically detects the user's AV and displays its performance against other tested AVs using a weighted scoring framework. Results are visualized in a summary report that ranks each antivirus solution across

detection capability, system impact, and usability providing users with an accessible and comparative evaluation.

3.6 Implementation and Development Process

For the design process, the system was built around the three stages of testing which is the malware simulation tests (EICAR, Atomic Red Team), GoPhish and perfromance logging with modular automation for each stage. Each stage should be sequential and the results should be compiled and saved in the end of the test in the format of .csv. As for the comparision tool, the main idea is to create a frontend system to display all the data collected from the benchmark tool with the exception of it being able to detect the users installed AV through the usage of powershell.

Python will serve as the main coding language of the project. Using its ability to execute PowerShell commands, manage test sequences and control the logging tools. In order to create the three modules, the EICAR test file must be automatically generated using python scripting, once generated the script should automatically trigger powershell commands to place the EICAR file in the system where the script should then extract specific event IDs from Windows Event Viewer Logs to precisely record timestamps and event details. The python script can also parse file system activity logs to capture the movement of the test files and to record the exact time taken by the AV to detect and quarantine the test files. Event logs and system performance metrics can be collected from Windows Perfromance Monitor which are exported through the usage of powershell commands which are triggered by the python script.

Next, the python script sends API calls such as REST API to the HQ VM in order initiate the phishing test where the once sent the HQ VM will send a phishing email to the system where the script will then automatically click on the link to trigger the AV's response. The data will be collected the same way as the previous stage using the same logic. Lastly, once the script has successfuly verified that both tests were completed, the script should be able to launch Atomic Red Team tests by triggering powershell commands to simulate the attack techniques provided by the Atomic Red Team. Since Invoke-Atomic is a powershell framework specially developed for executing Atomic Red Team tests, it is perfect for this project and can seamless work alongside it as it will be bundled in together with the program so it can run locally. As for the comparison tool, the GUI will be coded in python to create a lightweight and straightfoward program to compare

antivirus performance. A script will also be written to use powershell to detect the current AV utilized by the system.

For the testing strategies, each module will be developed seperately and then combined to create a functional program. Each module will be tested in isolation in the VM and once each module has been confirmed to be functional, only then will development move to combining the three modules. Error checks will be implemented to ensure that phishing links and EICAR is working and to ensure that each data collected is correct. As for the usability testing, it will evaluated manually based on the structured rubric derived from TAM and ISSM where the score will be manually inserted alongside the automated scores.

Once the program has successfully been tested and is working as intended in the HQ VM, it will be emailed to each VM running on the Proxmox homelab where it will then be executed one by one in the system. Each VM should produce a report in the form of a .csv file which contains the exported logs and the processed scores. All the .csv files will be compiled and processed and visualized using a python GUI application.

CHAPTER FOUR

SYSTEM ANALYSIS

4.1 Introduction

This chapter focuses on the planned system design and analysis for the proposed project, which will consist of two software solutions which are the benchmark tool and the comparison tool. The main objective of this chapter is to focus on how the system addresses the current situation and meets the functional and non-functional requirements, and to analyze whether the planned output is capable of solving the existing issues. The analysis will focus on the intended design and improvement strategy rather than actual performance data.

The current process of testing antivirus software is often manual, technical, and time-consuming. The benchmark tool will use Python and PowerShell scripting to automate malware scenarios using EICAR, Atomic-Invoke and GoPhish. This will address the issue of fragmented testing methods by ensuring the process is automated and recording antivirus responses through Windows Event Viewer. The relevant data will include factors such as detection speed, file movement and keylogging behavior. The comparison tool will be a hard-coded Python software that will display the data collected from the benchmark tool through a graphical interface, solving the current difficulty in comparing antivirus software for non-technical users.

4.2 System Requirements Overview

This section will analyze the functional and non-functional requirements which were established in Chapter 3, with emphasis on how these requirements are intended to address current problems. This section will also evaluate whether the planned system components which is the benchmark tool and the comparison tool are capable of solving the limitations of the current situation and achieving the project objectives.

4.2.1 Functional Requirements

Executing simulated malware such as EICAR test files, Invoke-Atomic Red Team scripts and phishing simulations using GoPhish automatically addresses the current open source lack of safe, standardized and automated testing methods. Coordinating all simulations using Python to call

PowerShell solves the problem of running multiple tools separately. Capturing antivirus responses such as file quarantine and alert logs through Windows Event Viewer overcomes the problem of manually searching for scattered logs. Logging the collected data in a structured format ensures results are consistent for easier evaluation.

The comparison tool will not be dynamic and will be hard coded to display the data from the benchmark tool. This addresses the current issue where users have to manually interpret technical test reports by instead providing a simplified view.

4.2.2 Non-Functional Requirements

The non-functional requirements of the project relate directly to the project scope and objectives. Automation is prioritized to reduce the need for technical knowledge and avoid manual intervention. Accuracy ensures the results remain consistent, solving the problem of unreliable manual testing outcomes. Safety is achieved by using harmless but recognized testing methods such as EICAR and Atomic Red Team payloads instead of actual malware, avoiding the risks of dangerous files. Consistent data logging and reliable execution across different configurations address the issue of inconsistent results in current testing processes.

4.3 System Design Evaluation

The design of the benchmark tool emphasizes automation and simplicity. The only interaction required is a single start button that initiates the testing process. The sequential design begins with the EICAR test, followed by a GoPhish phishing simulation, and then a series of Atomic Red Team attack simulations. Each operation is contained in its own module, which can be triggered, monitored and logged independently. This modular design addresses the current difficulty of adding or removing test cases, making it easier to expand in future designs.

Alongside each module, a monitoring component will capture real-time antivirus responses using logs from Windows Event Viewer, PowerShell and detection alerts. These are then passed to the report module, which generates results displayed at the end of the process and available as a downloadable file.

The comparison tool is designed as a lightweight software that detects the active antivirus on the system and highlights it in a GUI comparison table. The table will display metrics such as detection rate, response time and feature support with Windows Defender as the baseline. This addresses the current challenge of manual antivirus comparison, offering a clear and accessible view for non-technical users.

While weaknesses such as requiring a consistent environment may affect portability, the design still fulfills the main objectives of automation and accurate data generation.

When compared to the current available benchmark tools available, the proposed benchmark and comparison tool focuses less on large-scale technical scoring and more on automation and accessibility. Industry wide benchmarks provide rigorous detection statistics but remain complex and targeted towards industry professionals whereas this projects tools aims to simplify and open source the process with an emphasis on automation and modularity making it easier for enthusiast to conduct their own testing, while the comparison tool simplifies interpretation of results for non-technical users. This approach prioritizes accessibility over exhaustive test coverage acknowledging that while the project does not replace professional labs, it fills the gap between academic research, industry benchmarks and practical decision making for everyday users.

4.4 Usability Analysis

The antivirus benchmark tool system is designed for ease of use and minimal intervention. Both tools consist of a single start button in the benchmark tool, it initiates the entire process without complex menus or configurations, and in the comparison tool, it automatically detects the current antivirus. This addresses the current problem of complex user interfaces in antivirus testing tools.

The GUI layout will be clean and uncluttered, showing results clearly at the end of execution. The comparison tool is intended for non-technical users who lack the expertise or time to manually test antivirus software. The benchmark tool, while primarily for controlled environments, can still be used by cybersecurity enthusiasts with minimal setup.

4.5 Anticipated Challenges and Limitations

Challenges include automating phishing simulations and malware tests in a way that reliably

triggers antivirus responses, as different AV software may react differently. Limited access to premium antivirus software restricts the comparison scope. For the comparison tool, designing a clear and simple GUI to present the data is also a challenge.

4.6 Future Improvements

Since the project is in the design phase, improvements are based on the limitations of current antivirus testing methods. The modular design of the benchmark tool will overcome the inflexibility of current tools by allowing more test modules to be added. This removes the problem of fixed and limited testing capabilities. The benchmark tool's design also addresses the issue of manual data entry into comparison tools by allowing for future integration with an online database. This would make updates automatic instead of manual. Another improvement is expanding testing to other operating systems, addressing the limitation of current Windows-only solutions.

CHAPTER FIVE

SYSTEM DESIGN

5.0 Introduction

This chapter will build upon the established analysis of the design and provide a blueprint of the proposed system. Using the findings conducted in chapter 4, the main goal of this chapter is to analyze the benchmark tool and the comparison tool from a system design point of view with the help of the analysis done in the last chapter.

The architecture, workflow and design diagrams will be presented in this chapter that will explain and present how the tools will work together to achieve the projects objectives. As mentioned before, the focus of the system design will be to focus on usability, automation and accuracy.

5.1 System Architecture & Modules

The benchmark tool will be designed with the main purpose of simulating controlled malware attacks to capture AV responses and log the corresponding results. The tool consists of four main modules which are the malware simulation module, data capture module, logging and storage module and lastly the reporting module. The malware simulation module purpose is to safely generate malware tests which consist of EICAR file, Atomic Red Team tests and GoPhish phishing simulation. The data capture module will aim to pull detection logs using windows event viewer. The logging and storage module will compile and export the logs into .csv file and lastly the reporting module will generate the compiled results which will then be used in the comparison tool. The flow of the system will consist of the simulation tests then data capture of each test, storing the data captured then lastly generating the report. The design choice emphasizes modularity as each module allows updating the simulation without interrupting the logging logic, allowing additional testing modules to be added without a problem.

Next, the main purpose of the comparison tool is to display the data collected from the benchmark tool and display the performance of each tested AV in a way that is easy for users to compare their antivirus against other antivirus solutions. The main focus of the tool is to highlight the difference

between built in solutions against third party solutions so that users can establish which one suits their needs. The comparison tool will consist of the GUI module which will be the user interface for viewing the data collected, the antivirus detection module which is a simple query to detect the currently active AV used by the user's system and lastly the data display logic which presents the data collected into a comparison table. Simply put, the benchmark tool generates the dataset, and the comparison table will use the dataset to present an easy-to-understand table for users to determine their antivirus performance against built in antivirus and other third-party antivirus. Separation of tool is a intended design choice as it allows a safe and easily updated tool without having to risk users' system with the usage of the benchmark tool.

5.2 Interface Design

The design principle that the system focuses on is minimal clicks, clean GUI and a consistent style for both tools. The benchmark tool is designed with full automation and minimal user interaction to minimize any human error. The tool is designed with a single start button which is the only interaction the user has with the tool until the report is generated. The same principle is also applied to the comparison tool where the user only has to click start then the tool will automatically detect the users AV and present the comparison table for the user to see. With this design choice, the target of minimal clicks is achieved. Both tools will also have consistent styling and a clean GUI for users to see the progress of the tools and the presentation of the data.

The planned UI layout of the benchmark tool mainly consists of a status page where users can see in real time the progress of the tool as it runs each test with it ending at a report page summarizing the findings for the user and providing the user with the exported dataset. A clear focus on making the design for enthusiast users is implemented to make sure that most of the logic is done behind the scenes and the layout remains consistent regardless of any additional upgrades done to the system if needed. As for the comparison tool on the other hand, it is designed with an emphasis on non-technical user, meaning an easy and straightforward design aimed at presenting the data in a way that anyone can understand and process it.

5.3 Data Design

The system's data flow is designed to ensure autonomous and efficient capture, processing and retrieval of results from the malware simulation tests. The flow begins when the benchmark tool

first executes automated simulations like the atomic red team tests and the phishing campaign via GoPhish. Data is then collected from the simulations by utilizing monitoring scripts through Windows Event Viewer and PowerShell. The data will consist of values such as detection time, response time, and file handling actions. Once the benchmark has been completed, the data is then stored and ready to be imported into the comparison tool to be displayed in a graphical interface. The comparison tool utilizes the data collected to display the results in a graphical interface while also highlighting the user's active antivirus and comparing it with the other tested antivirus solution based on the values collected and through the weighted scoring method.

Although both systems do not utilize a complex database, the benchmark tool aims to provide data which the comparison tool will then refine into clear and informative tables. Each record will include attributes such as Test ID, Antivirus Name, Free/Paid and the weighted score. Users can then view the details of the test with attributes such as detection time for each test, success rate for malware simulation test (Atomic Red Team) and antivirus behavior and response to each test. Relationships between records ensure that all data linked to a single test session can be easily accessed with strict validation rules such as rejecting empty fields to maintain data integrity.

5.4 Technology, Security & Performance

The system will be developed primary using Python due to its flexibility, rich library ecosystem, and compatibility to work with PowerShell which makes it the best choice for automation and GUI-based applications on windows platforms. The benchmark tool will combine the usage of Python with PowerShell scripting to trigger antivirus simulations, collect event logs, and automate Windows-specific tasks efficiently.

The comparison tool on the other hand will be a hardcoded Python application utilizing frameworks such as Tkinter or pySimpleGUI for building a lightweight and intuitive graphical interface. The usage of Python allows the system to maintain compatibility across various Windows environments. Data will be saved through .csv format to keep the system lightweight but data will be manually added into the comparison tool. Additional technologies such as Invoke-Atomic Red Team payloads and GoPhish will serve as testing and benchmarking toos without actually deploying any actual malicious software. Hardware requirements is aimed at being

minimal due to how lightweight the system will be with any modern windows machine capable of running Python 3.x and PowerShell expected to be able to handle the system effectively.

While the system does not require a dedicated server, it can be integrated down the road with cloud-based solutions that can store and retrieve tests results centrally. This will allow the benchmarks to be automatically stored in a dedicated server where the comparison tool will sync with it and update the table in real time.

Testing elements such as the EICAR files, Atomic Red Team tests and phishing simulations will only be conducted in an isolated environment such as virtual machines to avoid impacting the host system. This ensures that security is considered when developing the system as anything involving the act of testing malware could be a risk to the host system therefore developing a clean and safe comparison tool for users to use is essential to ensure that their system has no risk of being affected by any malware test.

As for scalability, the system is designed in a way that more modules can be added without interfering with the logging logic. Users can also tinker with the system if they want to due to the modularity making it possible to curate their own tests.

5.5 Error Handling, Deployment & Maintenance

Error handling in the system will focus on catching any exceptions and errors during the logging module. Aside from that, the system should be able to make sure that any failed processing or malware testing does not interrupt the process and the overall workflow. An error message should appear detailing which module encountered an error in the report phase. For the benchmark tool, invalid or missing test files will trigger error messages and skip to the next module instead of stopping the entire testing process. This ensures that the error is isolated based on the modules. The comparison tool on the other hand, will handle missing or corrupted benchmark data by displaying an error message. Since the logging module stores data locally, this allows easy trouble shooting, debugging and refining test scenarios in future updates.

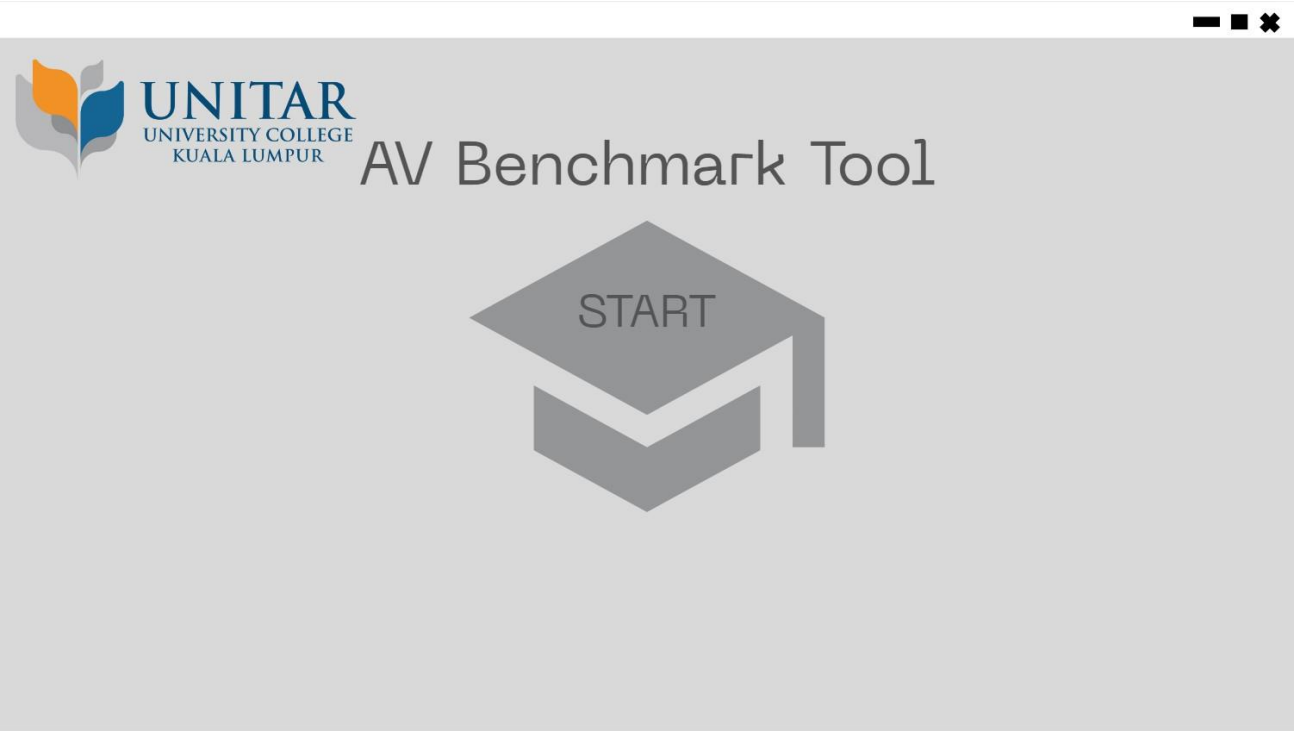
Deployment will initially be local, with only the comparison tool being available for public use. The setup is straightforward thanks to it being hard coded and lightweight therefore, it requires no additional dependencies beyond what is bundled with the executable. Separately, the benchmark tool is available for usage in closed test environment with the user being responsible for the usage

of the benchmark tool. Maintenance of the two tools will involve periodic updates to testing scripts, adding antivirus detection scenarios and GUI refinements. This modular approach will allow updates to individual components without affecting the entire system.

5.6 Design Challenges & Summary

There are several anticipated challenges, mainly in ensuring that all antivirus consistently respond to the benchmark tool as different vendors and AV configurations setting differ for each tool meaning that each AV reacts differently to simulated malware or phishing scenarios. Another challenge is the comparison of premium antivirus solutions which also narrows the initial comparison scope. Lastly, the usage of Python and PowerShell which is perfect for automation actually limits the compatibility to only windows environment. Overall, the proposed system design meets the projects core functional and non-functional requirement.

5.7 UI Design Mockup



Initializing First Phase....

Generating EICAR

Initializing First Phase....

EICAR Generated Successfully

Data has been logged successfully

Initializing First Phase....

EICAR Generated Successfully

AV Not Responding

Returning Back to Menu.....

Initializing Second Phase....

Beginning Phishing Attack...

Please do not use your mouse or keyboard, The tool is now controlling the desktop to perform tests.

Initializing Second Phase....

Phishing Attack Complete

Data has been logged successfully

Initializing Third Phase....

Running Test 1 of 10: Atomic Red Team Simulation...

Initializing Third Phase....

Atomic Red Team Simulation Completed

Data has been logged successfully

AV Benchmark Report

Antivirus Tested: Microsoft Defender

Test Completed: August 22, 2025, 10:45 AM

OVERALL SUMMARY

FINAL SCORE
78.5 / 100

SCORE BREAKDOWN (Weighted)
Detection (50%): 85
Performance (30%): 70
Usability (20%): 90

DETAILED TEST RESULTS

Detection Analysis (Overall Score: 85)

Test Scenario	Result	Detection Time
Malware Detection (EICAR)	DETECTED	450 ms
Phishing Simulation (GoPhish)	DETECTED	1200 ms
Threat Simulation (Atomic Red Team)	9/10 Detected	
Test T1548.002	Detected	800 ms
Test T1548.004	Detected	200 ms
Test T1543.003	Detected	700 ms
Test T1546.003	Detected	900 ms
Test T1541.003	Detected	800 ms
Test T1549.003	Not Detected	0 ms
Test T1540.003	Detected	400 ms
Test T1544.003	Detected	1200 ms
Test T1544.303	Detected	400 ms
Test T1544.033	Detected	600 ms

System Performance Metrics (Overall Score: 70)

Metric	Recorded Value (Average)
CPU Usage	15%
RAM Usage	250 MB
Disk I/O (Read/Write)	12 MB/s / 5 MB/s

Usability Evaluation

Score derived from Technology Acceptance Model (TAM) rubric. Manually inputted by user.

Back to Menu

UNITAR
UNIVERSITY COLLEGE
KUALA LUMPUR

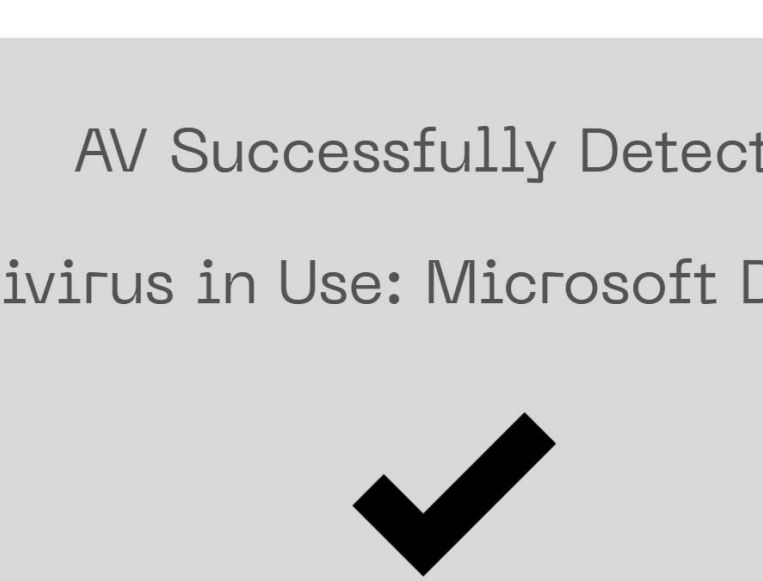
AV Comparison Tool

START

Detecting Installed AV....

AV Successfully Detected

Antivirus in Use: Microsoft Defender



AV Benchmark Report

Antivirus Tested: Microsoft Defender

Test Completed: August 22, 2025, 10:45 AM

OVERALL SUMMARY

FINAL SCORE
78.5 / 100

SCORE BREAKDOWN (Weighted)
Detection (50%): 85
Performance (30%): 70
Usability (20%): 90

DETAILED TEST RESULTS

Detection Analysis (Overall Score: 85)

Test Scenario	Result	Detection Time
Malware Detection (EICAR)	DETECTED	450 ms
Phishing Simulation (GoPhish)	DETECTED	1200 ms
Threat Simulation (Atomic Red Team)	9/10 Detected	
Test T1548.002	Detected	800 ms
Test T1548.004	Detected	200 ms
Test T1543.003	Detected	700 ms
Test T1546.003	Detected	900 ms
Test T1541.003	Detected	800 ms
Test T1549.003	Not Detected	0 ms
Test T1540.003	Detected	400 ms
Test T1544.003	Detected	1200 ms
Test T1544.303	Detected	400 ms
Test T1544.033	Detected	600 ms

System Performance Metrics (Overall Score: 70)

Metric	Recorded Value (Average)
CPU Usage	15%
RAM Usage	250 MB
Disk I/O (Read/Write)	12 MB/s / 5 MB/s

Usability Evaluation

Score derived from Technology Acceptance Model (TAM) rubric. Manually inputted by user.

Back to Menu

CHAPTER SIX

CONCLUSION

6.0 Conclusion

In conclusion, the proposed antivirus benchmark and comparison system aims to provide a structured, automated and user-friendly approach to evaluating antivirus solutions performance using safe and controlled testing methods such as EICAR, Atomic Red Team tests and GoPhish phishing simulations. By using a combination of Python and PowerShell, the system is able to achieve full automation in executing tests, capturing antivirus responses and presenting results in an accessible format. The modular design also ensures flexibility in terms of enhancements and usability with it being suitable for both cybersecurity enthusiasts in controlled environments and everyday users seeking quick antivirus comparisons. Although limitations exist with the fact that premium antivirus products locks certain features behind a paywall and windows specific dependencies, the design remains adaptable, scalable and secure laying a strong foundation for future iterations and broader platform compatibility.

REFERENCES

Echegu, D. A., & Agwu, C. O. (2024). Advancements in computer virus protection: From origins to future trends. Kampala International University. https://www.researchgate.net/publication/385892274_Advancements_in_Computer_Virus_Protection_From_Origins_to_Future_Trends

Nilupul, S. A. (2024). Evolution and impact of malware: A comprehensive analysis from the first known malware to modern-day cyber threats. Sri Lanka Institute of Information Technology (SLIIT).https://www.researchgate.net/profile/AmithSenewirathna/publication/383177164_Evolution_and_Impact_of_Malware_A_Comprehensive_Analysis_from_the_First_Known_Malware_to_Modern-Day_Cyber_Threats/links/66bf90282ff54d6c9ed752c9/Evolution-and-Impact-of-Malware-A-Comprehensive-Analysis-from-the-First-Known-Malware-to-Modern-Day-Cyber-Threats.pdf

Delaney, J. (2020). The effectiveness of antivirus software (Master's capstone project, Utica College). ProQuest Dissertations Publishing. https://www.proquest.com/openview/d3ff27e1e773c8dd36e7746e64567702/1?pq-origsite=gscholar&cbl=44156

Cruz, B. (2024, November 8). *2024 Antivirus Trends, Statistics, and Market Report*. Security.org. https://www.security.org/antivirus/antivirus-consumer-report-annual/

Shinder, T. W., Diogenes, Y., & Shinder, D. L. (2013). Windows Server 2012 security from end to edge and beyond: Architecting, designing, planning, and deploying Windows Server 2012 security solutions. Syngress.

Baker, E. (2018). Security in antivirus software: A comparative study of efficiency and performance [Bachelor's thesis, University of Skövde]. DiVA Portal. https://www.diva-portal.org/smash/record.jsf?pid=diva2%3A1211982

Ferdous, M. S., Mollah, M. B., Rahman, A., Islam, S., Hasan, S. R., & Bai, G. (2023). A review of state-of-the-art malware attack trends and defense mechanisms [Research article]. ResearchGate. https://www.researchgate.net/publication/375462049_A_Review_of_State-of-the-Art_Malware_Attack_Trends_and_Defense_Mechanisms

Lindemulder, G., & Forrest, A. (2023, December 20). *Next-generation antivirus*. Ibm.com. https://www.ibm.com/think/topics/next-generation-antivirus

Endpoint Protection - Next-Gen Antivirus Technology. (2024, February 16). ThreatDown by Malwarebytes. https://www.threatdown.com/products/endpoint-protection/

denisebmsft. (2025, May 7). *Microsoft Defender for Endpoint - Microsoft Defender for Endpoint*. Microsoft.com. https://learn.microsoft.com/en-us/defender-endpoint/microsoft-defender-endpoint

Drummond, K. E., & Murphy-Reyes, A. (2018). *Nutrition research : Concepts and applications.* Jones & Bartlett Learning.

Real-World Protection Test February-May 2025. (2025, June 16). AV-Comparatives. https://www.av-comparatives.org/tests/real-world-protection-test-february-may-2025/